

T.C. MARMARA ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



Teknoloji Fakültesi

MİKROİŞLEMCİLER

2025-2026 BAHAR DÖNEMİ

LABİRENTTEN KAÇIŞ

8086 Assembly ile Grafik Tabanlı Labirent Oyunu Proje Raporu

Ders Sorumlusu:

Arş. Gör. Merve Hazan İşcan

Arş. Gör. ASAF KARATAŞ

Hazırlayan:

Adı Soyadı: Fatih KAYA

Öğrenci No: 171424530

<https://github.com/fat1hkaya/Assembly-8086-Maze-Game>

1. GİRİŞ VE PROJE AMACI

Bu proje raporu, Intel 8086 mikroişlemci mimarisi temel alınarak "emu8086" simülatörü üzerinde Assembly dili ile geliştirilen "**Labirentten Kaçış**" adlı metin tabanlı (ASCII) oyunun tasarım, geliştirme ve çalışma prensiplerini detaylandırmaktadır.

Projenin temel amacı, kullanıcının klavye girdileri aracılığıyla bir karakteri kontrol ederek, 20x20 boyutlarındaki karmaşık bir labirentin başlangıç noktasından çıkış noktasına (X) ulaşmasını sağlamaktır. Oyun, kullanıcıya toplam 180 saniyelik (3 dakika) bir zaman sınırı sunmakta olup, bu süre zarfında artan zorluk derecesine sahip üç farklı haritayı geçmesini beklemektedir. Karakterin duvarlardan geçememesi (çarpışma algılama - collision detection) ve oyun akışının gerçek zamanlı bir sayaç ile kısıtlanması, projenin temel oyun mekaniklerini oluşturmaktadır.

Assembly gibi donanım seviyesine son derece yakın, düşük seviyeli bir dilde böyle bir interaktif oyun geliştirmek, yüksek seviyeli dillerde soyutlanan (abstracted) birçok bilgisayar bilimleri kavramının pratik olarak anlaşılmasını sağlamaktadır. Bu proje kapsamında;

- Mikroişlemci yazmaçlarının (AX, BX, CX, DX, SI, DI vb.) veri iletimi ve matematiksel işlemlerde efektif kullanımı,
- Bellek segmentasyon mantığı ve yığıt (stack) yönetimi,
- Video belleğine (Video RAM) doğrudan erişim ile ekran manipülasyonu,
- Donanım ve yazılım kesmelerinin (Interrupts) oyun döngüsü içerisine entegre edilmesi hedeflenmiştir.

2. TEKNİK ALTYAPI VE KULLANILAN KESMELER (INTERRUPTS)

Proje, 16-bit x86 mimarisini simüle eden emu8086 ortamında, standart 80x25 çözünürlüklü metin modunda (Text Mode 03h) çalışacak şekilde dizayn edilmiştir. Oyunun akıcı bir deneyim sunması ve donanım kaynaklarının doğrudan kontrol edilmesi için çeşitli sistem kesmelerinden ve doğrudan bellek erişim (DMA benzeri) yöntemlerinden faydalanılmıştır.

2.1. Video Belleği (VGA RAM) ve Doğrudan Erişim

Standart DOS işletim sistemlerinde ekrana karakter yazdırmak için genellikle kesmeler (örn: INT 21h AH=09h) kullanılsa da, bu yöntem bir oyun döngüsü (game loop) içerisinde ekranın tamamını sürekli güncellemek için oldukça yavaştır ve ekranda titremelere (flickering) yol açar.

Bu projede yüksek performans sağlamak amacıyla, ekran kartının metin belleği olan **0B800h** segmentine doğrudan erişim sağlanmıştır. ES (Extra Segment) yazmacı bu bellek adresine

yönlendirilerek, ekrandaki her bir hücreye doğrudan yazma işlemi gerçekleştirilmiştir. Metin modunda ekrandaki her bir karakter bellekte 2 byte yer kaplar; birinci byte görüntülenecek ASCII karakterini, ikinci byte ise karakterin renk ve arka plan (attribute) bilgisini tutar.

2.2. Sistem Kesmeleri (Interrupts)

Oyunun girdi/çıkışı (I/O) işlemleri ve zamanlayıcı mekanizması BIOS ve DOS kesmeleri kullanılarak inşa edilmiştir:

- **INT 10h (Video Hizmetleri):** Oyunun başlangıcında AH=00h ve AL=03h parametreleriyle çağrılarak ekran çözünürlüğünün 80x25 metin moduna ayarlanması ve ekranın eski verilerden tamamen temizlenmesi sağlanmıştır.
- **INT 16h (Klavye Hizmetleri ve Asenkron Okuma):** Oyunun en kritik mekanizmalarından biri asenkron (non-blocking) girdi okumasıdır. Eğer klavye okuması için standart beklemeli kesme kullanılsaydı, oyuncu bir tuşa basana kadar zaman sayacı da duracaktı. Bunun önüne geçmek için öncelikle AH=01h kesmesi ile klavye arabelleğinde (buffer) bir tuş basımı olup olmadığı kontrol edilmiştir. ZF (Zero Flag) bayrağı kontrol edilerek, eğer tuşa basılmamışsa program akışı zamanı saymaya devam eder. Eğer bir tuş tespiti yapılırsa, AH=00h kullanılarak bu tuş arabellekten alınır ve karakterin yönlendirilmesi sağlanır.
- **INT 21h (DOS Gelişmiş Hizmetleri):** * **Sistem Saati Okuma (AH=2Ch):** Oyun içi 180 saniyelik geri sayım sayacını oluşturmak için sistemin gerçek zamanlı saati (RTC) kullanılmıştır. Bu kesme çağrıldığında DH yazmacı saniye bilgisini döndürür. Eski saniye verisi (PREV_SEC) ile anlık saniye verisi karşılaştırılarak, her saniye değişiminde TIME_LEFT değişkeni bir azaltılır.
 - **Karakter Dizisi Yazdırma (AH=09h):** Oyun kazanıldığında veya süre bittiğinde ekrana gösterilecek olan "TEBRIKLER" veya "KAYBETTİN" şeklindeki veri segmentinde (Data Segment) tanımlanmış \$ sonlu mesajların ekrana basılması için kullanılmıştır.

3. VERİ YAPILARI VE HARİTA TASARIMI

Assembly dilinde, yüksek seviyeli dillerdeki gibi çok boyutlu diziler (matrisler) veya gelişmiş veri tipleri donanımsal olarak yerleşik halde bulunmaz. Tüm veriler doğrusal (linear) bir bellek adres alanında tutulur. Bu nedenle, projedeki 20x20 boyutlarındaki labirent haritaları, tek boyutlu byte dizileri (Data Byte - DB) olarak modellenmiştir.

3.1. Harita Dizilerinin Bellek Yerleşimi

Her bir harita, 400 byte'lık (20 satır x 20 sütun) bir blok olarak Data Segment (Veri Bölütü) içerisinde tanımlanmıştır. Harita tasarımında üç farklı sayısal sabit kullanılmıştır:

- **1 (Duvar):** Karakterin geçemeyeceği engeller. Ekranı açık mavi renkli 178 ASCII kodlu gölgeli blok karakteri olarak çizdirilir.
- **0 (Yol):** Karakterin hareket edebileceği boş alanlar. Ekranı boşluk (Space) karakteri olarak çizdirilir.
- **2 (Çıkış Noktası):** Labirentin bitiş noktası. Ekranı kırmızı renkli X karakteri olarak çizdirilir.

3.2. Bellek Optimizasyonu ve Dinamik Harita Yükleme

Oyunda üç farklı seviye (MAP1, MAP2, MAP3) bulunmasına rağmen, çarpışma algılama ve çizim algoritmalarının bu haritalar için ayrı ayrı yazılması büyük bir kod israfına (code bloat) yol açacaktır. Bunun yerine **CURRENT_MAP** adında 400 byte'lık, içi sıfırlarla dolu (400 DUP(0)) dinamik bir tampon bellek (buffer) alanı tanımlanmıştır.

Oyuncu yeni bir seviyeye geçtiğinde, LOAD_LEVEL prosedürü devreye girer ve hedef haritanın bellek adresi SI (Source Index), tampon haritanın adresi ise DI (Destination Index) yazmaçlarına yüklenir. x86 mimarisinin güçlü string taşıma komutu olan **REP MOVSB** kullanılarak, CX (Counter) yazmacındaki değer (400) kadar byte, kaynak adresten hedef adrese donanım hızında kopyalanır. Taşıma işleminin doğru yönde yapılması için CLD (Clear Direction Flag) komutu ile yön bayrağı ileri okuma (forward) moduna ayarlanmıştır. Bu mimari, oyun motorlarındaki "Scene Loading" (Sahne Yükleme) mantığının donanım seviyesindeki karşılığıdır.

3.3. Oyun İçi Durum Değişkenleri

Oyun motorunun anlık durumunu (State) koruyabilmesi için Word (16-bit, DW) ve Byte (8-bit, DB) veri tiplerinde değişkenler kullanılmıştır:

- **PLAYER_X** ve **PLAYER_Y**: Oyuncunun matris üzerindeki güncel koordinatları.
- **NEW_X** ve **NEW_Y**: Klavye girdisi alındığında oyuncunun gitmek istediği "aday" koordinatlar (Çarpışma testi için).
- **TIME_LEFT** (16-bit): 180'den geriye sayan ana zamanlayıcı.
- **CURRENT_LEVEL**: Oyunun hangi aşamada olduğunu takip eden state değişkeni.

4. MATEMATİKSEL MODELLER VE TEMEL ALGORİTMALAR

Assembly 8086 donanımında tüm matematiksel işlemler doğrudan işlemci yazmaçları üzerinde gerçekleştirilir. Projede, iki boyutlu matris uzayından, tek boyutlu doğrusal bellek uzayına veri dönüştürmek için iki temel formül kullanılmıştır.

4.1. Video Belleği Ofset (Offset) Hesaplama Algoritması

Metin modunda ekran 80 sütun (X) ve 25 satırdan (Y) oluşur. Karakterin ekranda doğru konuma basılabilmesi için (X, Y) koordinat çiftinin, video belleği olan 0B800h segmentinin başlangıcından ne kadar uzaklıkta (ofset) olduğunun hesaplanması gerekir. Ekranda her karakter bir byte ASCII kodu ve bir byte renk niteliği (attribute) olmak üzere toplam 2 byte kapladığından şu formül kullanılmıştır:

$$\text{Offset} = (Y \times 80 + X) \times 2$$

Assembly'de çarpma işlemi (MUL) gerçekleştirilirken, hedef operand yazmacının kapasitesine göre işlemci sonucu otomatik olarak DX:AX ikilisine yazar. Bu donanımsal davranış, özellikle döngü içerisinde indeksleme yaparken veri ezilmesine (data corruption) neden olabilir. Bu durumu izole etmek için **GET_SCREEN_OFFSET** isimli güvenli bir hesaplama alt programı (procedure) yazılmıştır. Y değeri 80 ile çarpılmadan önce işlemde kullanılacak yazmaçlar yığıta (PUSH) atılarak korunmuş, MUL işlemi tamamlanıp ofset değeri DI (Destination Index) yazmacına aktarıldıktan sonra yığıttan geri (POP) alınmıştır.

4.2. Harita Çarpışma Tespiti (Collision Detection) İndeks Algoritması

Oyuncunun hareket etmek istediği yeni (NEW_X, NEW_Y) koordinatlarının, CURRENT_MAP isimli 400 elemanlı tek boyutlu dizide hangi indekse denk geldiğini bulmak için şu formül kullanılmıştır:

$$\text{Index} = Y \times 20 + X$$

Bu işlem **GET_MAP_INDEX** prosedürü içerisinde gerçekleştirilmiş ve sonuç BX (Base Register) yazmacına kaydedilmiştir. Ardından işlemci doğrudan bellek adreslemesi (CURRENT_MAP[BX]) yaparak ilgili hücredeki baytı okur. Okunan değer 1 (Duvar) olması durumunda, oyuncunun aday koordinatları reddedilerek mevcut pozisyonunda kalması sağlanır.

4.3. Zamanlayıcı ve Basamak Ayırıştırma Algoritması

180 saniyelik zamanlayıcının ekrana üç haneli (Yüzler, Onlar, Birler) olarak basılabilmesi için matematiksel bir ayırıştırma algoritması kullanılmıştır. Ekrana sayısal değerler doğrudan basılamaz; ASCII karakter formatında (Örn: 0 için 48, 1 için 49) basılmalıdır.

Bu işlem için, kalan zaman değeri önce 100'e (DIV CX) bölünerek yüzler basamağı elde edilmiş, ardından bölümünden kalan (DX) değer 10'a bölünerek onlar ve birler basamağı ayırıştırılmıştır. Elde edilen her rakamın üzerine 48 ('0' karakterinin ASCII karşılığı) eklenerek doğrudan görüntü belleğindeki sağ üst köşe (140. ofset) adresine yazdırılmıştır.

5. KOD MİMARİSİ VE ALT PROGRAMLARIN (PROSEDÜRLERİN) ANALİZİ

Projenin yazılım mimarisi, modülerlik ve sürdürülebilirlik ilkeleri göz önünde bulundurularak alt programlara (procedures) ayrılmış bir "Durum Makinesi" (State Machine) olarak tasarlanmıştır. Bu sayede ana oyun döngüsü (Main Game Loop) sadeleştirilmiş ve karmaşık işlemler izole edilmiştir.

5.1. Ana Döngü (GAME_LOOP) ve Girdi İşleme

MAIN prosedürü programın giriş noktasıdır. Veri segmenti (DS) belleğe yüklendikten ve ekran video moduna (80x25 Metin) geçirildikten sonra ilk harita LOAD_LEVEL prosedürü ile belleğe çekilir. Ardından oyun, sürekli çalışan bir döngü olan GAME_LOOP bloğuna girer.

Bu döngünün en önemli görevi, zamanlayıcıyı güncellemek ve kullanıcı girdilerini işlemektir. Asenkron klavye okuma (INT 16h, AH=01h) sayesinde sistem kilitlenmeden (blocking olmadan) çalışır. Kullanıcıdan 'W', 'A', 'S', 'D' yön tuşlarından biri geldiğinde, karakterin mevcut koordinatları (PLAYER_X, PLAYER_Y) geçici aday değişkenlere (NEW_X, NEW_Y) kopyalanır ve basılan tuşa göre sadece aday değişkenler modifiye edilir (Örn: 'W' tuşu için NEW_Y değeri DEC komutu ile bir azaltılır).

5.2. Çarpışma Bloğu (CHECK_COLLISION) ve Seviye Geçişi

Değiştirilen aday koordinatlar, hareketin yasal olup olmadığını test etmek üzere CHECK_COLLISION bloğuna gönderilir. Burada GET_MAP_INDEX prosedürü çağrılarak matris indeksi hesaplanır.

- Eğer hedef hücrenin değeri 1 (Duvar) ise, hiçbir işlem yapılmadan GAME_LOOP'un başına dönülür ve hareket reddedilir.
- Eğer değer 2 (Çıkış) ise, NEXT_LEVEL bloğu tetiklenir. CURRENT_LEVEL değişkeni INC komutu ile artırılır ve sıradaki harita belleğe yüklenir.
- Eğer değer 0 (Yol) ise, hareket onaylanır. Önce CLEAR_PLAYER çağrılarak karakterin eski konumundaki izi silinir, aday değişkenler asıl oyuncu değişkenlerinin üzerine yazılır ve DRAW_PLAYER ile karakter yeni konumunda ekrana çizilir.

5.3. Ekran Çizim Prosedürleri (DRAW_MAP, DRAW_PLAYER)

Oyunun görsel motoru, doğrudan video belleğine erişen bu prosedürler üzerine kuruludur:

- **DRAW_MAP:** 400 byte'lık tampon haritayı baştan sona tarar. X ve Y sayacı tutarak matrisin sonuna gelindiğinde (CMP BX, 20) alt satıra geçer. LODSB (Load String Byte) komutunu

kullanarak bellekten (SI yazmacının gösterdiği adresten) baytları son derece hızlı bir şekilde AL yazmacına çeker ve karakterleri ekrana yerleştirir.

- **DRAW_PLAYER / CLEAR_PLAYER:** Oyuncunun koordinatlarını GET_SCREEN_OFFSET alt programına gönderir. Dönen ofset değerine göre ES : [DI] adresine gülen yüz (02h) veya boşluk (' ') karakterini basar.

6. KARŞILAŞILAN ZORLUKLAR, ÇÖZÜMLER VE SONUÇ

Donanım seviyesinde geliştirilen bu proje sürecinde, yüksek seviyeli dillerde karşılaşılmayan spesifik mikroişlemci ve bellek yönetimi sorunlarıyla karşılaşılmış ve uygun çözümler geliştirilmiştir.

6.1. Yazmaç Ezilmesi (Register Corruption) Problemi

Assembly dilinde çarpma işlemi olan MUL komutu, operandı ne olursa olsun işlemi AX üzerinden yapar ve taşma durumları için sonucu her zaman DX : AX ikilisine (32-bit sonuç) yazar. Harita çizimi sırasında Y koordinatını tutmak için DX yazmacı kullanılıyordu. Video bellek ofseti hesaplanırken MUL komutu çalıştırıldığında işlemci DX yazmacındaki Y koordinat verisini donanımsal olarak sıfırlıyor (eziyor) ve bu durum tüm haritanın hatalı bir şekilde sadece 0. satıra (ilk satıra) çizilmesine yol açıyordu.

Çözüm: Bu donanımsal kısıtlama, yığıt (Stack) bellek yönetimi kullanılarak aşılmıştır. MUL komutu çalıştırılmadan hemen önce kritik yazmaçlar PUSH komutuyla yığita itilerek koruma altına alınmış, matematiksel işlem bittikten sonra POP komutu ile veriler geri yüklenerek veri bütünlüğü (data integrity) sağlanmıştır.

6.2. Ekran Belleği Hizalama (Alignment) ve Taşma Sorunu

Oyunun geliştirilme aşamasında, emu8086 simülatörünün pencere boyutunun manuel olarak genişletilmesi (Örn: 80x25'ten 106x43'e) durumunda haritanın ekranda parçalandığı ve koordinatların kaydığı gözlemlenmiştir.

Çözüm: Video belleği mantığında satır atlama işlemi fiziksel değil, mantıksaldır. Offset hesaplama formülünde ($Y * 80 + X$) doğrudan 80 sabitine bağlıdır. Bu sebeple kodun başına `MOV AX, 0003h / INT 10h` kesmesi eklenerek sistemin video modunun her şartta standart 80x25 çözünürlüğe kilitlenmesi ve ekranın sıfırlanması sağlanmış, böylece donanımsal hizalama sorunu giderilmiştir.

6.3. Sonuç

Bu proje; düşük seviyeli dillerde bellek manipölasyonunun, yığıt (stack) yönetiminin, kesmelerin (interrupts) asenkron kullanımının ve doğrudan donanım (video bellek) erişiminin ne kadar güçlü ancak bir o kadar da hataya açık bir yapı olduğunu açıkça göstermiştir.

Geliştirilen sistem, saniyede onlarca kez ekran güncellemesi yapabilen, hafızayı optimum seviyede (tampon harita kullanımı ile) değerlendiren ve donanım hızında çalışan eksiksiz bir ASCII tabanlı oyun motoru haline gelmiştir. Gelecekteki çalışmalarda bu projeye düşman yapay zekası (A* Pathfinder gibi algoritmalarla çalışan hareketli nesneler), puan/skor hesaplama mekanizmaları ve daha gelişmiş çok sayfalı bellek kullanımı (Page flipping) eklenerek oyun deneyimi zenginleştirilebilir.